

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №4 по курсу
«Операционные системы»**

Студент: Черноус Алексей Тимофеевич
Группа: М8О-209Б-23
Вариант: 1
Преподаватель: Миронов Евгений Сергеевич
Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Содержание

1. Репозиторий
2. Постановка задачи
3. Общие сведения о программе
4. Общий метод и алгоритм решения
5. Исходный код
6. Демонстрация работы программы
7. Выводы

Репозиторий

https://github.com/HGRaicer/MAI_OS/tree/main/lab3

Постановка задачи

Цель работы

Приобретение практических навыков в:

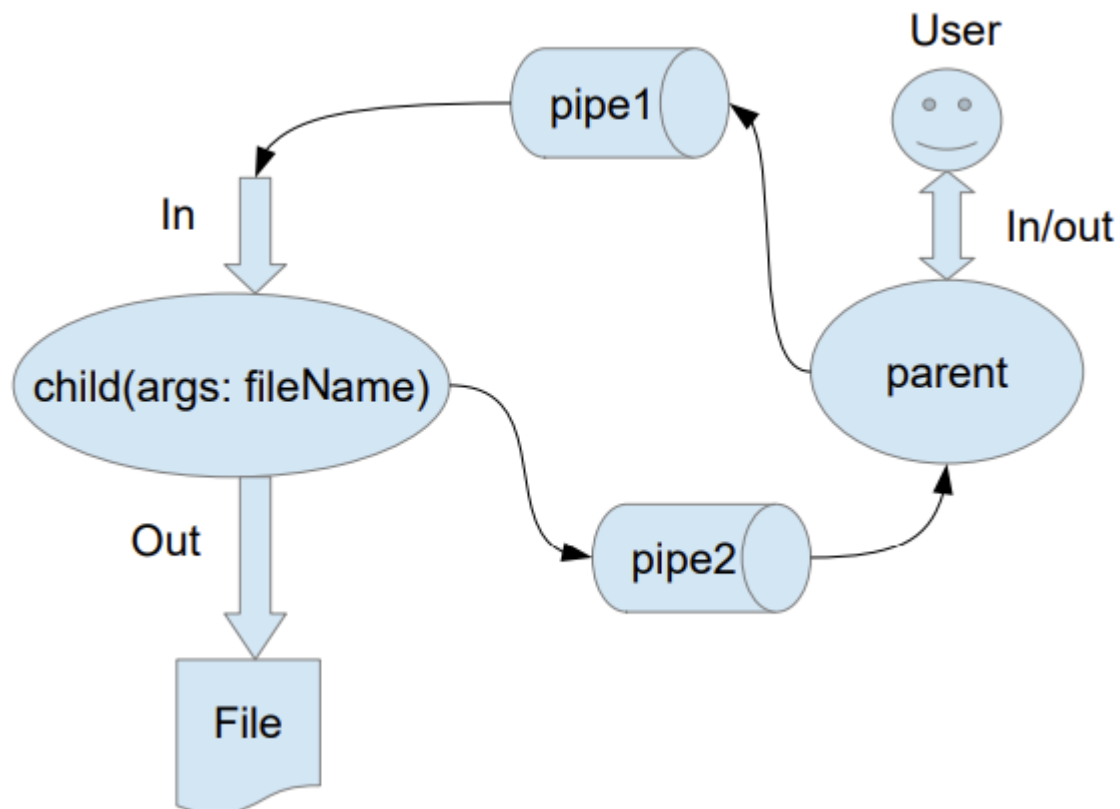
- Освоение принципов работы с файловыми системами
- Обеспечение обмена данными между процессами посредством технологии «File mapping»

Задание

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решение задачи один или несколько дочерних процессов.

Взаимодействие между процессами осуществляется через системные сигналы/события и/или через отображаемые файлы (memory-mapped files).

Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.



1 вариант) Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс считает их сумму и выводит её в файл. Числа имеют тип `int`. Количество чисел может быть произвольным.

Общие сведения о программе

Программа компилируется из файла `main.c`. Также используется заголовочные файлы: `unistd.h`, `stdio.h`, `stdlib.h`, `fcntl.h`, `errno.h`, `sys/mman.h`, `sys/stat.h`, `string.h`, `stdbool.h`, `ctype.h`, `sys/wait.h`, `semaphore.h`. В программе используются следующие системные вызовы:

1. `shm_open` - создаёт/открывает объекты общей памяти POSIX.
2. `ftruncate` - обрезает файл до заданного размера.
3. `mmap`, `munmap` - отображает файлы или устройства в памяти, или удаляет их отображение.
4. `memset` - заполнение памяти значением определённого байта.
5. `close` - закрывает файловый дескриптор.
6. `execl` - запуск файла на исполнение.

Общий метод и алгоритм решения

Для реализации поставленной задачи необходимо:

1. Изучить работу с отображением файла в память(`mmap` и `munmap`).
2. Изучить работу с процессами(`fork`).
3. Создать 1 дочерний и 1 родительский процесс.
4. В каждом процессе отобразить файл в память, преобразовать в соответствии с вариантом и снять отображение(`mmap`, `munmap`).

Исходный код

parent.c

```
#include <stdio.h>
```

```
#include <string.h>
```

```

#include <stdlib.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/stat.h>

int main(int argc, char* argv[]) {
    const char* back_name = "Lab3.back";
    unsigned perms = S_IWUSR | S_IRUSR | S_IRGRP | S_IROTH;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    char* input_data = NULL;
    size_t size = 0;
    printf("Enter numbers separated by spaces: ");
    getline(&input_data, &size, stdin);

    size_t input_len = strlen(input_data);
    if (input_data[input_len - 1] == '\n') {
        input_data[input_len - 1] = '\0';
        input_len--;
    }

    size_t map_size = strlen(argv[1]) + 1 + input_len + 1;

```

```

int fd = shm_open(back_name, O_RDWR | O_CREAT, perms);
if (fd == -1) {
    perror("SHM_OPEN");
    free(input_data);
    exit(EXIT_FAILURE);
}

if (ftruncate(fd, map_size) == -1) {
    perror("FTRUNCATE");
    close(fd);
    free(input_data);
    exit(EXIT_FAILURE);
}

char* memptr = mmap(NULL, map_size, PROT_READ | PROT_WRITE,
MAP_SHARED, fd, 0);
if (memptr == MAP_FAILED) {
    perror("MMAP");
    close(fd);
    free(input_data);
    exit(EXIT_FAILURE);
}

snprintf(memptr, map_size, "%s|%s", argv[1], input_data);

pid_t cpid = fork();
if (cpid == -1) {
    perror("FORK");
    munmap(memptr, map_size);
}

```

```

    close(fd);
    free(input_data);
    exit(EXIT_FAILURE);
}

if (cpid == 0) {
    munmap(memptr, map_size);
    close(fd);
    execl("./build/child", "child", NULL);
    perror("EXECL");
    exit(EXIT_FAILURE);
} else {
    int status;
    wait(&status);
    free(input_data);
    munmap(memptr, map_size);
    close(fd);

    if (WIFEXITED(status) && WEXITSTATUS(status) == 0) {
        printf("Child process completed successfully.\n");
        exit(EXIT_SUCCESS);
    } else {
        fprintf(stderr, "Child process failed.\n");
        exit(EXIT_FAILURE);
    }
}
}

```

child.c

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int check(char ch) {
    return (ch >= '0' && ch <= '9') ? 0 : 1;
}

float calc_func(char* data, int size) {
    int result = 0;
    char buffer[10] = {0};
    int jj = 0;

    for (int k = 0; k < size; ++k) {
        if (check(data[k]) == 0) {
            buffer[jj++] = data[k];
        } else if (jj > 0) {
            buffer[jj] = '\0';
            result += atoi(buffer);
            jj = 0;
        }
    }
}
```



```

    return (float)result;
}

int main() {
    const char* back_name = "Lab3.back";
    unsigned perms = S_IWUSR | S_IRUSR | S_IRGRP | S_IROTH;

    int map_fd = shm_open(back_name, O_RDWR, perms);
    if (map_fd < 0) {
        perror("SHM_OPEN");
        exit(EXIT_FAILURE);
    }

    struct stat statbuf;
    if (fstat(map_fd, &statbuf) == -1) {
        perror("FSTAT");
        close(map_fd);
        exit(EXIT_FAILURE);
    }

    size_t map_size = statbuf.st_size;

    char* memptr = mmap(NULL, map_size, PROT_READ | PROT_WRITE,
MAP_SHARED, map_fd, 0);
    if (memptr == MAP_FAILED) {
        perror("MMAP");
        close(map_fd);
        exit(EXIT_FAILURE);
    }
}

```

```

close(map_fd);

char filename[20] = {0};
size_t readed_data_id = 0;

// Разделяем имя файла и данные
for (size_t i = 0; i < map_size; i++) {
    if (memptr[i] != '|') {
        filename[i] = memptr[i];
    } else {
        readed_data_id = i + 1;
        break;
    }
}

char* input_data = memptr + readed_data_id;
size_t input_size = map_size - readed_data_id;

float result = calc_func(input_data, input_size);

FILE* file = fopen(filename, "w");
if (!file) {
    perror("FOPEN");
    munmap(memptr, map_size);
    exit(EXIT_FAILURE);
}

```

```

fprintf(file, "%.2f\n", result);
fclose(file);

munmap(memptr, map_size);
return 0;
}

```

Демонстрация работы программы

```

rai@rai-laptop:~/cods/os/lab3$ make run FILE=output.txt NUMBERS="1 2 3 4 5"
mkdir -p build
gcc -Wall -g -o build/parent src/parent.c
mkdir -p build
gcc -Wall -g -o build/child src/child.c
echo "1 2 3 4 5" | valgrind ./build/parent output.txt
==1970== Memcheck, a memory error detector
==1970== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1970== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1970== Command: ./build/parent output.txt
==1970==
Enter numbers separated by spaces: Child process completed successfully.
==1970==
==1970== HEAP SUMMARY:
==1970==   in use at exit: 0 bytes in 0 blocks
==1970== total heap usage: 3 allocs, 3 frees, 5,240 bytes allocated
==1970==
==1970== All heap blocks were freed -- no leaks are possible
==1970==
==1970== For lists of detected and suppressed errors, rerun with: -s
==1970== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

Выводы

В Си помимо механизма общения между процессами через pipe, также существуют и другие способы взаимодействия, например отображение файла в память, такой подход работает быстрее, за счет отсутствия постоянных вызовов read, write и тратит меньше памяти под кэш. После отображения возвращается void*, который можно привести к своему указателю на тип и обрабатывать данные как массив, где возвращенный указатель – указатель на первый элемент.